General Notes:

- The only ecalls you will need for this lab are **those located in the table in the lab instructions** and **exit**
- Is recommended that all immediately loaded numbers that aren't used for ecalls (not a7) should be written in hex (begins with "0x").
  - But their equivalent decimal values (without "0x") will also be accepted.
- Do not add anything to the offset when referencing file descriptors (should always be 0)
- If you end up with a single black pixel in the corner, check your branch condition, as it is likely an off-by-one error

Part 2 Outline:

```
open the file
```

this ecall takes 1 input:

- a *.string* containing the path to the file being loaded
- a flag indicating read-only (0) or write-only (1)

this ecall returns:

- a file descriptor **that you need to keep track of**
  - Recall this from the lab instructions:
    Note: you only get the descriptor from the "open" ecall, and others will overwrite it.
    Be sure to store the result in a saved register after opening the file so you can use it later.

```
read the header
```

this ecall takes 3 inputs:

- a file descriptor **that you should have already started keeping track of**
- an address to the target memory address ("header")
- how many bytes to read (size of "header")

this ecall returns:

- the length of data read from the target memory address **that will overwrite the file descriptor if you didn't save it**

```
load the constants needed to loop through the bitmap's pixel data
```

- the starting address (in hex) of where you want to load data from
  - $startAddr = 0x10010000$
  - you'll only need this once to initialize a starting point for your loop

- since the RARS bitmap tool frame buffer uses s0, this should be loaded into s0

- the address (in hex) that you want to stop loading data from
  - $endAddr = ((h \times w) - 1) \times memPixelSize + startAddr$
  - where:
    - $memPixelSize = 0x04 = 4_{10}$ is the size (in bytes) per pixel data in memory for the display
    - $(h \times w)$ is the height (h) and width (w) of the image in pixels.
  - you can hardcode the result; you are **not required to calculate it using RARS** (https://www.wolframalpha.com/input?i=%28%28w-x%29%2F3%29-1%29*4%2By+where+w%3D196662%2C+x%3D54%2C+y%3D268500992).

- you can get $(h \times w)$ from the file header as in the lab instructions. Remember that the file header gives you the total size of the bitmap file as follows:
  - $bmpFileSize = [(h \times w) \times bmpPixelSize] - headerSize$
  - where:
    - $bmpPixelSize = 0x03 = 3_{10}$ is the size (in bytes) per pixel data in the bitmap file
    - $headerSize = 0x36 = 54_{10}$ is the size (in bytes) of the header

```
loop through pixel data
```

begin a loop for reading and drawing pixel data

```
    read pixel data into the buffer
```

similar to "read the header" above

this ecall takes 3 inputs:

- a file descriptor **that you should have continued to keep track of**
- an address to the target memory address ("buffer")
- how many bytes to read (size of "buffer")

this ecall returns:

- the length of data read from the target memory address **that will overwrite the file descriptor if you didn't save it**

```
    load pixel data from buffer and store it in the RARS bitmap tool frame buffer
```

once the above ecall has run, the target buffer ("buffer") will contain the pixel data to display in the bitmap tool

- load the pixel from the buffer ("buffer") into a temporary register
- store that temporary register in the RARS bitmap tool frame buffer
  - recall the instruction used to **store** "t0" in "**s0**" in Part 1
  - the offset will be controlled by the loop, not manually offset from 0 as in Part 1

```
    increment the address where we will draw the next pixel in the RARS bitmap tool frame buffer
```

recall that we need to iterate ahead by **4**, not 1

- this will point the address to the pixel we want to draw in the next iteration

```
    branch back to the start of the loop until the iterator meets or exceeds the address we want to stop loading data from
```

otherwise, exit the loop

after the loop exits...

```
close the file
```

this ecall takes 1 input:

- a file descriptor **that you should have been keeping track of**

```
exit the program
```

see the bottom of any prior RARS lab from this semester for an example of

Changes to Part 2 necessary to "flip" the image:

There are 2 approaches to this.

The easy version result is still mirrored horizontally. Technically, the hard version is the accurate way to load the image, but either will receive full credit if implemented correctly.

- Rotating the image 180 degrees (easY.t
  - swap your start address and end address
  - negate the iterator value
  - modify your branch condition logic accordingly

- Vertically mirroring the imag_!L(hardt
  - convert the loop to a pair of nested loops:
    - row by row (bottom to top): iterate every (i x w x $memPixelSize)$
      - column by column (left to right): iterate every (j x $memPixelSize) + startAddress$
        - $startAddr =$ OxlOOlOOOO
  - Note: you may be able to perform some algebra to simplify the computation (and faster performance).

Search entries or author Unread @>

Reply